Part 10 — Constraint-based modelling in practice: solving it yourself, COBRApy, reconstruction and MATLAB (weeks 35 to 38)

Part 2 chapter 13 set up S·v = 0 and said that stoichiometry constrains while measurements or an objective decide. These four weeks turn that into software you run. The tracker's route is COBRApy, and that is the right tool; this environment has no network access, so I could not install it. Rather than write four chapters of untested commands, I built a 40-reaction core model of *E. coli* central metabolism by hand and solved every analysis in the weeks' lists — FBA, flux variability, deletions, pFBA, shadow prices, quality control and gap filling — with `scipy.optimize.linprog`, and ran them.

That turns out to be the better teaching order. Writing the linear programme yourself makes it impossible to treat FBA as a black box: you see the equality constraints, the bounds, the objective vector and the duals, and when something is wrong you have to find it. While building the model I made three real errors, each of which a tool would have hidden. The chapters below keep them, because diagnosing them is the skill: a phosphate bookkeeping inconsistency that let an ATP-wasting cycle run free, a missing soluble transhydrogenase that silently blocked the whole TCA cycle and capped growth at less than half, and a vestigial exchange reaction that the blocked-reaction check in chapter 37 caught.

Where the chapters give COBRApy or MATLAB code, it is marked not run and should be checked against the current documentation. Where a number appears, it came out of the scripts in this part. The model is mine and not the published core model, so its numbers are close to but not identical with the tracker's reference values, and I say which is which.

Ch.	Week	Chapter
35	35	Flux balance analysis, written as the linear programme it is
36	36	Working with models: media, loops, pFBA, thermodynamics and reproducibility
37	37	Reconstruction and quality control: drafts, gap filling and what a check detects
38	38	MATLAB and the COBRA Toolbox: reading other people's code

Flux balance analysis, written as the linear programme it is

FBA is three lines of mathematics and a solver. Writing those three lines yourself once is worth a month of using a package, because everything that goes wrong later is a bound, a medium or an objective — never the mathematics.

The optimisation problem, in full

Part 2 chapter 13's balance plus bounds plus an objective is a linear programme:

```
maximise    cᵀv                  c selects the objective — usually the biomass reaction
subject to  S·v = 0              one equality per internal metabolite: production = consumption
            lb ≤ v ≤ ub          irreversibility, measured uptake rates, capacity limits
```

Three features of that statement carry the whole method. It is **linear**, so it solves in milliseconds even at genome scale, and the solution sits at a vertex of the feasible polytope. It needs **no kinetic parameters**: no K_M, no k_{cat}, no concentrations appear anywhere. And the price of that is what it cannot tell you — **no concentrations, no time, no regulation, and no statement that the cell will actually do this**. The objective is a hypothesis about what the cell optimises, as part 2 chapter 13 insisted; FBA returns a flux distribution that is *consistent* with the network, not one that is observed.

The model used throughout this part

Forty reactions and twenty-nine metabolites covering glycolysis, the pentose phosphate pathway, the TCA cycle, anaplerosis, respiration, the fermentative branches, nitrogen assimilation through glutamate dehydrogenase, maintenance, and a biomass reaction. The biomass coefficients are rounded from the published *E. coli* core model's, the P/O assumptions are 2.5 ATP per NADH and 1.5 per quinol, and maintenance is fixed at 8.39 mmol ATP·gDW^{-1}·h^{-1}. Glucose uptake is bounded at 10 mmol·gDW^{-1}·h^{-1}. Building it is the whole of the code; solving it is four lines:

```
import numpy as np
from scipy.optimize import linprog

S  = np.zeros((len(mets), len(rxns)))            # filled from a dict of {reaction: {metabolite: coeff}}
c  = np.zeros(len(rxns)); c[idx['BIOMASS']] = -1 # linprog minimises, so negate to maximise
res = linprog(c, A_eq=S, b_eq=np.zeros(len(mets)),
              bounds=list(zip(lb, ub)), method='highs')
growth = -res.fun
```

Rank and degrees of freedom first, because they say what the network leaves undetermined: 29 metabolites, 40 reactions, rank 29, so **11 degrees of freedom**. Eleven numbers must come from outside — the uptake bounds, the maintenance demand and the objective supply them, which is exactly part 2 chapter 13's three ways to close a system, now made concrete.

Growth, aerobic and anaerobic

With glucose at 10 and oxygen unconstrained, the model grows at **0.754 h^{-1}**; with oxygen set to zero, at **0.217 h^{-1}**, a ratio of 3.5. The tracker's reference values from the published core model are about 0.87 and 0.21, a ratio of about 4.2. Mine is lower aerobically because my lumped respiratory chain and my rounded biomass coefficients are not theirs; the anaerobic figure agrees closely. If you reproduce this with COBRApy on the real core model, expect the tracker's numbers, and treat a mismatch as a difference in medium, bounds or objective before suspecting the solver — which is chapter 38's rule as well.

The anaerobic solution closes its redox balance exactly as part 1 chapter 6 said it must: ethanol at 8.9, acetate at 5.4 and formate at 16.5 mmol·gDW^{-1}·h^{-1}, with pyruvate formate-lyase carrying 16.5. The fourfold drop in growth is the ATP yield difference, and the model reproduces it without being told anything about fermentation.

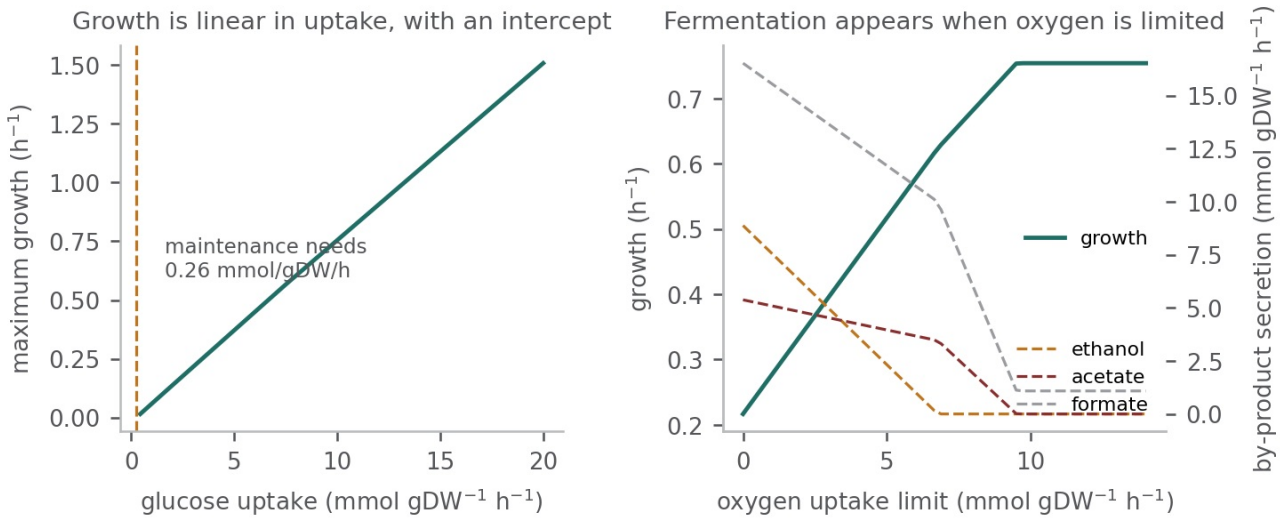


Figure 35.1 Left: maximum growth against the glucose uptake bound; the line is straight and meets zero at 0.26 mmol·gDW^{-1}·h^{-1}, the uptake that pays maintenance alone. Right: growth and by-product secretion as the oxygen bound is raised, showing the fermentative to respiratory transition. Computed with the hand-built model described in the text.

What the uptake sweep shows

Growth against glucose uptake is a straight line of slope 0.0754 gDW per mmol — the yield — and it crosses zero at 0.26 mmol·gDW^{-1}·h^{-1}. That intercept is the maintenance demand: the first fraction of a millimole of glucose buys no growth at all because 8.39 mmol of ATP must be paid first. Convert the slope and you get a biomass yield of **0.42 g per g of glucose**, which is in the region of measured aerobic yields for *E. coli* — a useful check that the biomass coefficients and the P/O assumptions are not wildly wrong.

The linearity is worth pausing on, because it is a property of linear programming and not of biology: with every bound except glucose non-binding, doubling the substrate doubles everything. Real cells deviate — overflow metabolism at high uptake (part 1 chapter 6), which plain FBA cannot predict because it has no proteome constraint, as part 4 chapter 51 explained.

The oxygen sweep, and a transition the model gets right

Raising the oxygen bound from zero to saturation walks the model through the fermentation-to-respiration transition:

O₂ uptake limit	Growth (h⁻¹)	Acetate	Ethanol	Formate
0	0.217	5.4	8.9	16.5
2	0.337	4.8	6.3	14.6
5	0.517	4.0	2.3	11.7
8	0.683	1.9	0	6.0
10 and above	0.754	0	0	1.1

Ethanol disappears first, then acetate, and growth saturates once oxygen stops limiting. Two honest observations. The model secretes a little formate even aerobically, because FBA knows nothing about pyruvate formate-lyase being oxygen-sensitive — a curation point, and the standard fix is to bound that reaction to zero in aerobic simulations. And the transition is a consequence of the redox balance alone: with limited oxygen the cell cannot reoxidise all its NADH through respiration, so it must dispose of electrons in a reduced product. No regulation is involved.

Deletions, and what a lethal prediction means

Knocking out each reaction in turn — setting its bounds to zero — gives ten lethal deletions out of thirty-nine:

Lethal: glucose and ammonium exchange, the glucose transporter, phosphofructokinase–aldolase, glyceraldehyde-3-phosphate dehydrogenase, enolase, transketolase, citrate synthase–isocitrate dehydrogenase, PEP carboxylase, glutamate dehydrogenase. **Growth-reducing:** oxygen exchange and NADH dehydrogenase to 0.22, phosphoglucose isomerase to 0.51, the transhydrogenase to 0.67, pyruvate dehydrogenase to 0.71.

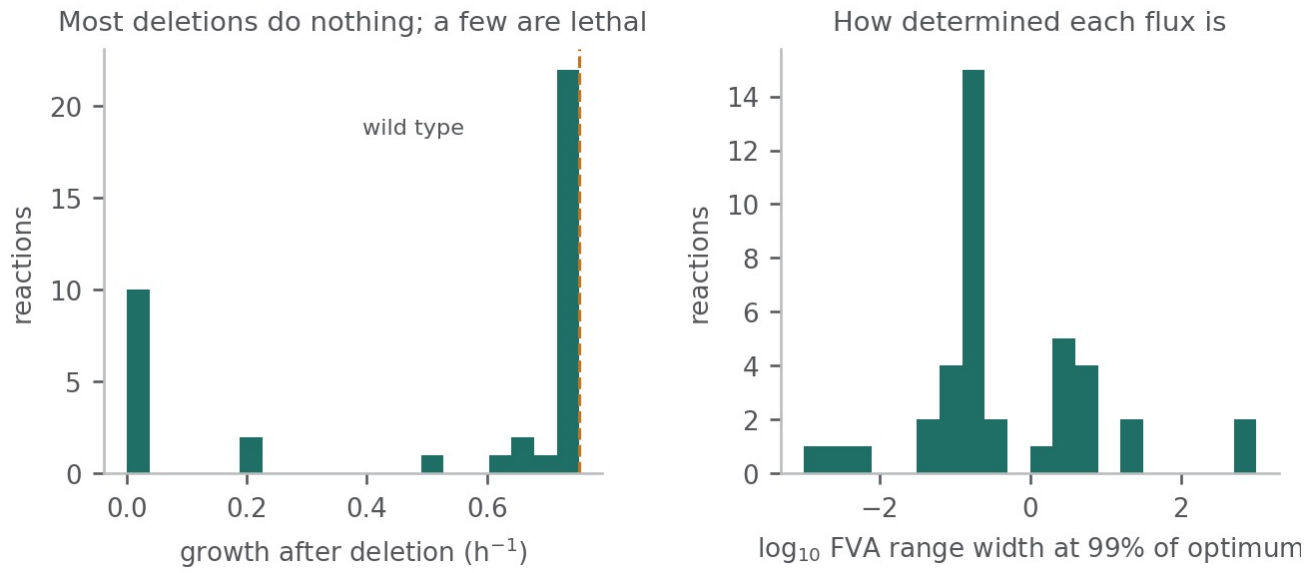


Figure 35.2 Left: the distribution of growth rates after single reaction deletions — most are silent, a few are lethal. Right: the distribution of flux-variability range widths at 99% of optimal growth, on a log scale; the two reactions at 1,000 are the loop of chapter 36. Computed for this module.

Check two against the biology, as the week asks. **Enolase** is lethal because there is no bypass from 3-phosphoglycerate to phosphoenolpyruvate in this network, and the tracker names exactly this class — the glycolytic trunk without a parallel route. **Phosphoglucose isomerase** is not lethal but costs a third of the growth rate, because carbon is forced through the oxidative pentose phosphate pathway and back, losing a carbon as CO_2 per turn — which is part 1 chapter 5's NADPH-versus-carbon trade appearing as a number. **Transketolase** is lethal in my model for a reason worth noticing: it is the only source of erythrose-4-phosphate, which biomass requires. In a real cell there are other routes, so this is a model artefact of the kind the tracker warns about — if a deletion you expected to be survivable is lethal, look for the route the organism has and the model does not, and vice versa.

Flux variability, and reading a wide range

Flux variability analysis fixes the objective at a fraction of its optimum and then minimises and maximises every flux in turn — two extra linear programmes per reaction. At 99% of optimal growth, the widest ranges in this model are:

Reaction	Minimum	Maximum	Reading
Succinate dehydrogenase	0	1,000	A loop — chapter 36
Fumarate reductase	0	1,000	The other half of the same loop
Transhydrogenase (NADH→NADPH)	7.5	26.0	Must carry flux, amount undetermined
Soluble transhydrogenase	0	18.1	Optional, and compensates the first
Maintenance ATP	8.4	14.4	At 99% of optimum, spare ATP can be burned
PEP carboxylase	6.4	12.3	Partly set, partly free

What a wide range means is precisely this: **the data and the objective do not determine that flux.** A reaction whose range spans zero may or may not carry flux in a solution that is just as optimal, so quoting its value from a single FBA solution is quoting an arbitrary choice. The honest way to report an FBA flux map is with its variability, which is why part 2 chapter 13 called flux variability analysis the honest way to report an FBA result.

Note the two transhydrogenases, which have wide ranges that compensate each other: the model is free to run NADH→NADPH and NADPH→NADH at various levels because only the net matters. That is the same structure as the loop below, in a milder form, and it is why *coupled* variability — how fluxes co-vary — is sometimes more informative than individual ranges.

Even at exactly 100% of the optimum, fourteen reactions still have a non-zero range, which means this model has alternative optima: several distinct flux distributions give identical maximum growth. Picking one and interpreting it biologically is the commonest misuse of FBA.

Shadow prices: what another millimole is worth

The dual variables of the equality constraints are the shadow prices — the increase in the objective per unit increase in

the availability of each metabolite. For this model at the aerobic optimum, glucose's shadow price is **0.0754 gDW per mmol**, which is exactly the slope of the uptake sweep. Oxygen's shadow price is **zero**, because oxygen is not binding: there is as much as the model can use, so another millimole buys nothing.

That contrast is the lesson. A shadow price of zero identifies a constraint that is not limiting, so effort spent relaxing it is wasted; a large shadow price identifies the resource to buy more of. Part 5 chapter 13 made this argument in the abstract; here it is one line of code — `res.eqlin.marginals` — and it tells you, for instance, that in an oxygen-limited process the shadow price of oxygen is what another unit of transfer capacity is worth in growth or product, directly comparable with the cost of supplying it.

The programme	max $c^T v$ s.t. $S \cdot v = 0$, $lb \le v \le ub$
Degrees of freedom	40 reactions − rank 29 = 11, closed by bounds, maintenance and the objective
Growth here	0.754 h^{-1} aerobic, 0.217 anaerobic (core model: ~0.87 and ~0.21)
Yield	slope 0.0754 gDW per mmol glucose = 0.42 g/g; intercept 0.26 mmol is maintenance
FVA	two LPs per reaction at a fixed objective; a wide range means undetermined
Shadow price	dual of the metabolite balance; zero means the constraint is not binding

Your FBA says a deletion is lethal and the experiment says the mutant grows. Give the four explanations in the order you would check them.

First, the medium: the model's exchange bounds may not match what the mutant was actually given, and a component present in the real medium — an amino acid, a vitamin — can bypass the blocked step entirely. This is the commonest cause and it costs nothing to check. Second, a missing route in the model: the organism has an isozyme, a bypass or a promiscuous enzyme that the reconstruction lacks, which is what my transketolase result above would be in a real cell; the test is to look for the compound's alternative route in a database and add it. Third, the gene–protein–reaction rule: a deletion of one gene may not remove the reaction if an isozyme shares it, so the lethality may be of the reaction and not the gene. Fourth, the biomass composition: if biomass demands a component the mutant can do without under those conditions — a specific lipid or cofactor whose coefficient is carried over from another strain — the lethality is an artefact of the objective.

The ordering matters because the first two are cheap and the fourth is laborious. And the result is worth having either way: a model that predicts a lethality the organism survives has told you there is a route you do not know about, which is a more useful outcome than agreement.

Three things people get wrong

- ‘FBA predicts the flux distribution.’ It returns one optimal vertex; fourteen reactions here have alternative optima at exactly the same growth rate.
- ‘No kinetic parameters means no assumptions.’ The objective, the bounds and the biomass composition are the assumptions, and they decide the answer.
- ‘The model says this deletion is lethal.’ It says so given this medium and this biomass equation. Check both before believing it.

Self-check — answer on paper first

1. Why does FBA need no kinetic parameters, and what does it therefore lose?

Because it imposes only the steady-state mass balances and capacity bounds, which are properties of the network's stoichiometry, and chooses among the resulting solution space by optimising a linear objective — none of which refers to rates as functions of concentration. What it loses is every question whose answer depends on concentration or time: it cannot predict a metabolite pool, a transient, a regulatory response, an intermediate's accumulation or the effect of an inhibitor, and it cannot tell you that the cell is actually at the optimum it computes.

2. The glucose shadow price is 0.0754 and the oxygen shadow price is zero. What would you spend money on?

Not oxygen transfer: at this operating point oxygen is not limiting, so another unit of k_La buys nothing in the model. Glucose is the binding resource, so the levers are the uptake rate itself and the efficiency with which the carbon is used. The caution is that the shadow prices are local to this solution — raise glucose far enough and oxygen will become binding, at which point its shadow price jumps from zero, which is exactly what the oxygen sweep shows from the other direction.

3. A flux you care about has an FVA range of −12 to +18 at 99% of optimal growth. What can you report?

That the network and objective do not determine either its magnitude or its direction, so no value from a single FBA

solution should be quoted for it. What you can report is the range itself, and what would narrow it: a measurement of that flux or of a flux coupled to it, a tighter uptake constraint, or a thermodynamic constraint fixing the direction if ΔG′ at physiological concentrations settles it — which is chapter 36's subject.

What to open a book for. Orth, Thiele & Palsson, "What is flux balance analysis?", Nat. Biotechnol. 28:245 (2010) — it is six pages and it is the paper to read before anything else. Then the COBRApy documentation and BiGG Models for the core model and iJO1366. Part 2 chapter 13 of this module is the derivation this chapter runs.

Working with models: media, loops, pFBA, thermodynamics and reproducibility

A thermodynamically impossible cycle can satisfy S·v = 0 perfectly. Finding one in your own model, and then removing it two different ways, is the exercise that teaches what a constraint-based model is and is not.

The object model, and building a network by hand

COBRApy represents a model as objects: a `Model` holding `Reaction`, `Metabolite` and `Gene` objects, with gene–protein–reaction rules linking genes to reactions. Building a reaction means creating it, setting bounds, and adding metabolites with signed coefficients — negative for substrates, positive for products. The code, not run here:

```
from cobra import Model, Reaction, Metabolite
m   = Model('toy')
A   = Metabolite('A', compartment='c'); B = Metabolite('B', compartment='c')
r   = Reaction('v2'); r.bounds = (0, 1000)
r.add_metabolites({A: -1, B: 1})
m.add_reactions([r])
m.objective = 'v4'
sol = m.optimize();  print(sol.objective_value)
```

The exercise is to build part 2 chapter 13's five-reaction toy network this way and check it against the hand solution — and the tracker's note about the usual culprit is right: if the answers differ, a sign convention on one reaction is reversed. My own experience building the 40-reaction model was the same lesson at larger scale, and the fastest way to find such an error is the carbon balance check of chapter 37, which catches a flipped sign immediately.

Media and bounds, and changes that undo themselves

The right way to set a carbon source is through the exchange reactions, which COBRApy exposes as `model.medium`: a dictionary of exchange reaction identifiers to uptake limits. Setting the medium rather than editing bounds by hand matters because it is the medium that defines what the organism is being offered, and because an exchange left open by accident is the commonest source of a model that grows on nothing. Switching between aerobic and anaerobic should be a change to the medium alone — removing oxygen — which in my model reproduces 0.754 and 0.217.

The context manager is the second habit:

```
with model:                      # everything inside is reverted on exit
    model.reactions.PGI.knock_out()
    print(model.slim_optimize())
print(model.slim_optimize())     # back to the wild type
```

Why this matters in a loop over hundreds of deletions is simple and brutal: without it, every knockout accumulates, so the second deletion is tested in the background of the first, the hundredth in the background of ninety-nine others, and the result is a plausible table of numbers that means nothing. When I ran the deletion scan in my own code I copied the bound arrays for each deletion for exactly this reason — `lb.copy()` inside the loop, never a mutation of the originals. The same discipline, by hand.

Loops: a cycle that satisfies the balances and cannot happen

My model contains succinate dehydrogenase (succinate → fumarate + quinol) and fumarate reductase (fumarate + quinol → succinate). Run them together at equal flux and the net conversion is nothing: every metabolite balances, S·v = 0 is satisfied exactly, and the pair can carry any flux at all up to whatever bound you set. The flux variability analysis of chapter 35 found exactly this — **both reactions range from 0 to 1,000**, the arbitrary upper bound, at 99% of optimal growth.

Why it cannot happen is thermodynamic, not stoichiometric. Going round the cycle returns every metabolite to its starting concentration, so the total ΔG around the loop is zero by construction; a net flux around a closed loop with zero driving force would be a perpetual motion machine. This is the class of error that part 2 chapter 14 called an internal cycle, and the point to absorb is that **S·v = 0 does not forbid it**: mass balance is indifferent to direction, and direction is set by free energy.

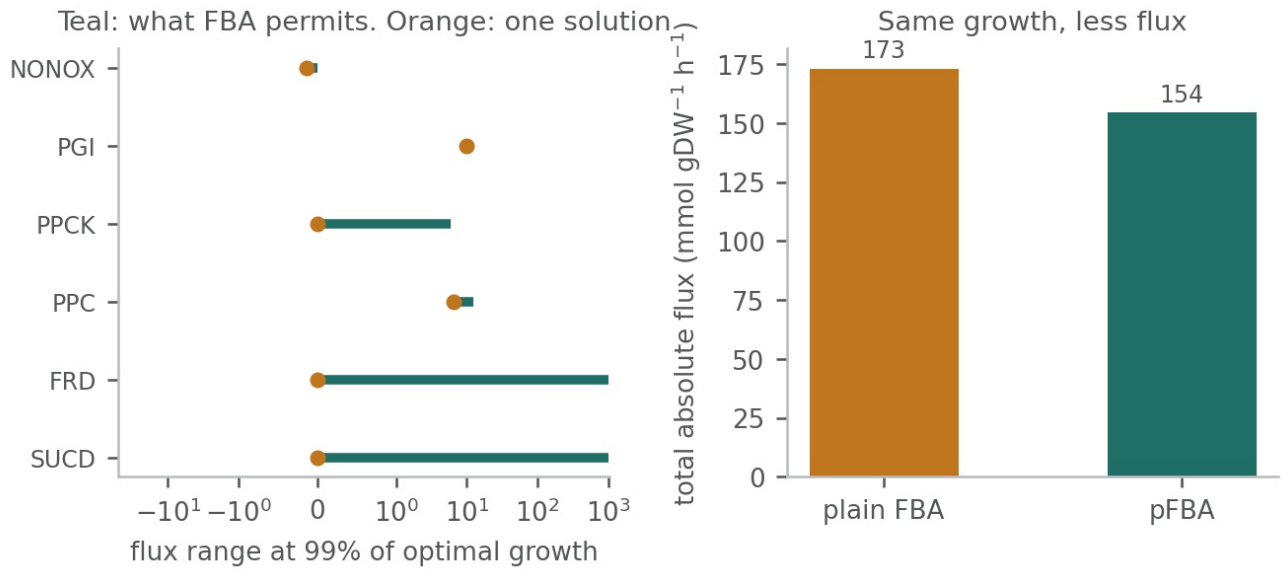


Figure 36.1 Left: for six reactions, the range flux variability analysis permits at 99% of optimal growth (bars, symmetric log scale) against the value one FBA solution happened to return (points). Succinate dehydrogenase and fumarate reductase run to the arbitrary 1,000 bound — the loop. Right: total absolute flux of a plain FBA solution and of a parsimonious one at the same growth rate. Computed for this module.

Two ways to remove it, and what each one is

Loopless FBA adds constraints that forbid net flux around any internal cycle, by requiring the existence of a thermodynamically consistent set of potentials. It is the cheap special case of thermodynamics-based flux analysis: it uses only the fact that a loop's ΔG is zero, with no actual thermodynamic data. The cost is that the problem becomes a mixed-integer programme, so it is much slower than FBA.

pFBA, parsimonious FBA, instead fixes the objective at its optimum and minimises the sum of absolute fluxes — the hypothesis being that the cell does not waste enzyme. It is a second linear programme, with each reaction split into forward and reverse parts so that the absolute value is linear. In my model it brings the total absolute flux from **172.9 to 154.4** at identical growth, and sets both halves of the loop to zero. It also changed the split between pyruvate dehydrogenase and formate-lyase, which is the alternative-optima point again: pFBA chooses among equally optimal solutions by a secondary criterion, which is a modelling assumption and not a fact.

pFBA removes a loop as a side effect of being parsimonious, and it does not *prove* the loop is absent; a loop that happened to lie on a minimal-flux solution would survive. Loopless FBA is the principled removal and pFBA is the cheap one, which is exactly how the tracker frames them.

From ΔG as a diagnostic to ΔG as a constraint

Part 2 chapter 14 used ΔG′ to decide whether a step can run. The step up is to put it into the optimisation as a bound. The procedure, which the week asks you to do with eQuilibrator values for a handful of core reactions:

- Get ΔG°′ for each reaction at the right pH and ionic strength, and compute ΔG′ at the concentration range the cell can hold — conventionally 1 µM to 10 mM, with cofactor ratios fixed, as in part 2 chapter 14.
- Where the sign of ΔG′ is the same across the whole feasible concentration range, the direction is determined: set the reaction's lower bound to zero (or its upper bound, for the reverse case). Where the sign can change, leave it reversible — that is an honest statement that thermodynamics does not decide it.
- Rerun and report what changed. In a well-constrained model the answer is often "nothing to the objective, something to the variability", which is the useful result: eliminating directions narrows FVA ranges and removes spurious solutions without changing the optimum.

Thermodynamics-based flux analysis does this systematically, carrying metabolite concentrations as variables and the ΔG of every reaction as a constraint linked to them. It is more expensive and it needs free energies for everything, which is why the loopless formulation — which needs none — is where most people stop.

The analysis functions, and exporting results

The functions the week lists all return DataFrames, which is the point: they are tables you join to other tables rather than printouts.

Function	What it returns	Watch for
`model.optimize()`	A solution object with fluxes, objective value, shadow prices and reduced costs	Use `slim_optimize()` in loops — it returns only the objective value and is much faster
`model.summary()`	Uptakes, secretions and the objective in a readable block	The fastest sanity check that the medium is what you think
`pfba(model)`	A parsimonious solution	It is a second optimisation with its own assumption
`loopless_solution(model)`	A solution with internal loops removed	Mixed-integer; slow at genome scale
`flux_variability_analysis(model, fraction_of_optimum=0.99)`	Minimum and maximum per reaction	Two LPs per reaction; the fraction is a choice you must report
`single_gene_deletion`, `double_gene_deletion`	Growth per deletion or pair	Gene, not reaction: isozymes mean the two differ

Export both the FVA and the deletion DataFrames to CSV and plot the deletion results as a histogram, as in figure 35.2 — the shape, with a spike at the wild-type rate and a few at zero, is what a deletion scan looks like in any model, and recognising a departure from it is how you spot a broken medium.

SBML, and what survives a round trip

SBML is the interchange format: an XML standard with an FBC extension that carries bounds, objectives and gene–protein–reaction rules. Write with `cobra.io.write_sbml_model`, read back, and compare reaction and metabolite counts, bounds and the objective. I could not test a round trip here — libsbml is not installed in this environment — so what follows is what to check rather than what I observed: counts and identifiers survive; bounds and the objective survive through the FBC extension; **annotations, notes and anything the writer put in free-text fields are the usual casualties**, along with any custom attribute your own script added to a reaction object. JSON round-trips more faithfully between COBRApy versions and is not portable to other tools, which is the trade. SBML is still what everyone uses because it is the only format every platform reads, which is chapter 38's reason for caring.

Escher draws a flux vector on a pathway map: load a core-model map, overlay your solution's fluxes as a dictionary of reaction identifier to value, and export. It is worth doing once for a solution you understand, because seeing where the flux goes catches errors that a table hides — a cycle running at high flux is obvious on a map and invisible in a sorted list.

Reproducibility, and a data management plan

The week's last session is the one that pays later. One notebook that runs top to bottom from a fresh kernel; pinned requirements; any randomness seeded; a README saying what the notebook shows and what the headline numbers are. Then the test that actually tests it: clone the repository into a clean directory and run it there. Everything else is a rehearsal.

A data management plan, which funders now require, fits on a page and answers five questions. **What data** — models, flux solutions, sequencing reads, fermentation time courses, each with its format and rough volume. **Where it lives** — raw data immutable and backed up, processed data derived by scripts that can be rerun, with the rule from part 9 chapter 32 that analysis reads from raw and never writes to it. **How it is named and versioned** — the identifier scheme, and version control for code and models. **What is deposited where on publication** — sequencing to a sequence archive, models to BioModels or a repository with a DOI, code to a tagged release. **How it meets the FAIR principles**: findable, through a persistent identifier and metadata; accessible, through an open repository; interoperable, through standard formats, which for a metabolic model means SBML with proper annotations; and reusable, through a licence and enough documentation for someone else to run it. Writing it before you are asked is cheap practice, and it also forces the identifier decisions early, which is when they are free.

Medium	`set the carbon source through exchange bounds, not reaction bounds`
Temporary change	`with model: – otherwise deletions accumulate through a loop`
Internal loop	`satisfies S·v = 0, forbidden by ΔG = 0 around a cycle; FVA shows ±bound`
pFBA	`fix the objective, minimise Σ|v|; here 172.9 → 154.4 at the same growth`
Thermodynamic bound	`if sign(ΔG′) is fixed over the feasible concentration range, fix the direction`
Round trip	`counts, bounds and objective survive SBML; annotations and notes often do not`

Your genome-scale model predicts a respectable growth rate and, on inspection, carries 500 mmol·gDW^{-1}·h^{-1} of flux around a three-reaction cycle. Does the growth prediction need to be thrown away?

No — but the flux map does. The growth rate is the objective value, and a loop carrying flux does not change it: the loop has no net conversion, so it neither consumes nor produces anything the biomass reaction needs. What is wrong is the *solution*, one of many optimal vertices, and any interpretation of individual fluxes from it. So the growth number stands, and every statement of the form "flux goes this way through that branch" has to be redone.

What to do, in order: run pFBA, which will almost certainly set the loop to zero and gives a flux map that is at least parsimonious; run flux variability analysis to see which other fluxes are undetermined, since a model with one loop usually has several; and if the loop involves reactions whose direction is thermodynamically decidable, fix those directions as bounds, which removes the loop permanently rather than solution by solution. Then check whether the loop is telling you about a curation error — a reaction duplicated in both directions, or a reaction whose reversibility was set to the default rather than considered, which is where most loops come from.

Three things people get wrong

- ‘If it satisfies the mass balances it is possible.’ A closed cycle balances perfectly and is thermodynamically forbidden.
- ‘pFBA is the correct solution.’ It is one choice among equally optimal ones, made by a secondary assumption about minimal enzyme.
- ‘I knocked out the genes in a loop over the list.’ Without a context manager the knockouts accumulate, and every row after the first is wrong.

Self-check — answer on paper first

1. Explain in two sentences why a thermodynamically impossible cycle satisfies S·v = 0.

Because S·v = 0 states only that each metabolite's production equals its consumption, and in a closed cycle every intermediate is both produced and consumed in equal measure, so the balance holds for any flux magnitude. Direction and feasibility come from free energy, which the stoichiometric matrix does not contain: going once round a cycle returns every concentration to its starting value, so the total ΔG is zero and a net flux would require work from nothing.

2. You need a flux map for a paper. Which analyses do you run, and what do you report?

FBA for the objective value; then pFBA or loopless FBA for a flux map without internal cycles; then flux variability analysis at a stated fraction of the optimum so that each reported flux carries a range. Report the objective, the fluxes from the parsimonious solution, and the FVA ranges alongside them, with the medium, the uptake bounds and the objective stated explicitly — and say which reactions had ranges wide enough that their values should not be interpreted.

3. Why is setting `model.medium` safer than editing reaction bounds directly?

Because the medium is the complete statement of what the organism is offered, so setting it makes every uptake explicit and any exchange you did not mention is closed; editing individual bounds leaves the rest at whatever they were, which is how a model ends up quietly consuming a nutrient you never intended to supply. It is also self-documenting: the medium dictionary printed in a notebook is the experiment's definition, whereas a sequence of bound edits has to be reconstructed by reading the code.

What to open a book for. The COBRApy documentation's building-a-model, simulating, deletions and FVA pages, which are the curriculum for this week, and its flux-analysis guide for pFBA and loopless FBA. BiGG Models for the core model and iJO1366 in both SBML and JSON, and Escher for the map. Part 2 chapter 14 is the chapter to reread before the thermodynamics section.

Reconstruction and quality control: drafts, gap filling and what a check detects

A quality report is only useful if you can say which failures would change a flux prediction and which are metadata. The way to learn that is to run the checks on a model you built yourself and see what they catch.

Where models come from

Manual reconstruction, as the Thiele and Palsson protocol describes it, is roughly a year of work: draw the draft from the genome annotation, then go reaction by reaction through the literature, fixing stoichiometry, cofactor usage, directionality, compartment assignment and gene associations, building a biomass equation from measured composition, and testing against growth phenotypes. The steps that cannot be automated are the ones that require judgement about evidence: deciding whether a paper's enzyme assay in a related organism justifies adding a reaction, choosing a direction when the thermodynamics are marginal, and deciding what the biomass equation should contain for *this* organism in *these* conditions.

Which is why automation exists. CarveMe starts from a universal reaction database and *carves* a species model out of it using genome evidence as scores — a top-down approach that produces a model that grows, fast. gapseq works bottom-up from annotation and pathway evidence, assembling reactions whose presence is supported and then filling gaps. A draft from either takes minutes to hours instead of a year and is correspondingly less trustworthy. And the first thing to check for your own organism is whether a curated model already exists in BiGG; for most non-model organisms it does not, which is the situation these tools are for.

Scale sets expectations. My hand-built core model has 40 reactions; an automated draft of a bacterial genome typically has of order a thousand to fifteen hundred; a curated genome-scale model of *E. coli* has around 2,600. A core model answers questions about central carbon and nothing else; a draft answers questions about capability; a curated model is the only thing that should be trusted for a gene-level prediction.

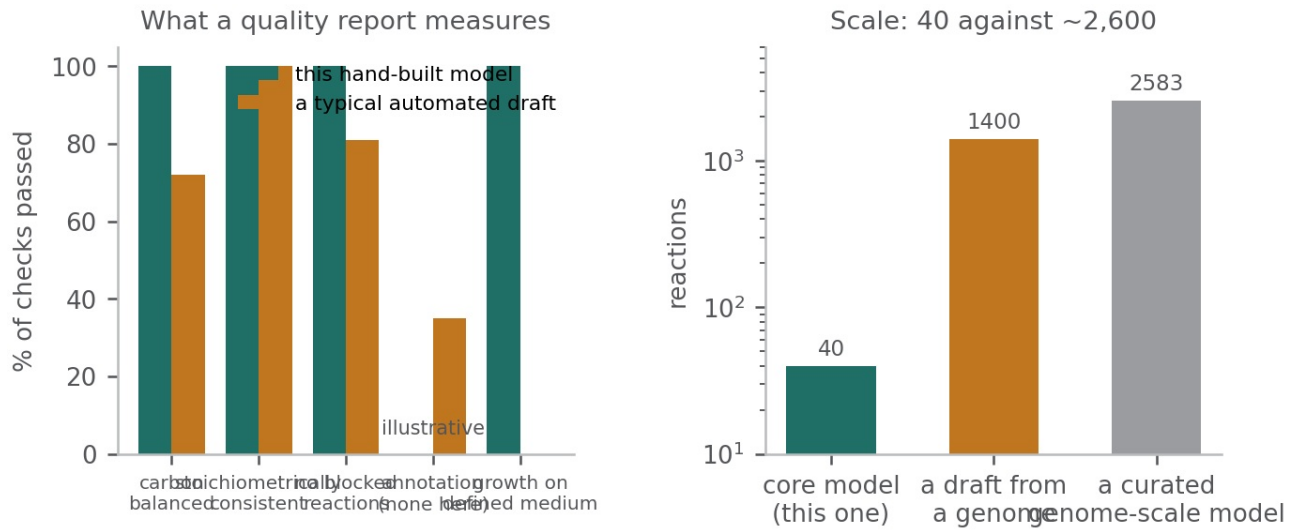


Figure 37.1 Left: the categories a quality report covers, with the results for the hand-built model of this part and illustrative values for a typical automated draft — the draft numbers are indicative, not measured, and are marked so. Right: reaction counts at three levels of reconstruction. Computed and drawn for this module.

Running the checks on my own model

MEMOTE scores a model on stoichiometric consistency, mass and charge balance, annotation coverage and SBML conformance, and produces an HTML report. I could not run it here, so I implemented the three checks that can change a prediction and ran them on the 40-reaction model. The results were not all what I expected, which is the point of the exercise.

Mass balance. Assign each metabolite its carbon count and check that every internal reaction's coefficients sum to zero in carbon. All **28 of 28** internal reactions balance, and the biomass reaction drains **70.87 carbons per unit of biomass** — which is not a failure but the definition of a sink, and is worth computing because it gives the carbon-limited ceiling directly: 10 mmol of glucose is 60 mmol of carbon, so no more than 0.85 h^{-1} of growth is possible however

efficient the energetics, against the 0.754 the model achieves. A real MEMOTE run checks full elemental formulas and charge, which catches protons and water that a carbon-only check misses.

Stoichiometric consistency asks whether there exists a strictly positive mass vector m with $S^{T}m = 0$ over the internal reactions — that is, whether a consistent set of molecular masses can be assigned so that every reaction conserves mass. **My model fails it.** The reason is instructive and not a mistake in the usual sense: ATP, NADH, NADPH and quinol are carried as bare currency metabolites with no ADP, phosphate or oxidised partner, so a reaction like NADH + ½O_2 → 2.5 ATP + H_2O cannot conserve mass under any assignment. Restricting the check to the carbon metabolites and excluding the currencies, the model **passes**, and the mass vector the solver returns is literally the carbon count of each metabolite — 6 for glucose and glucose-6-phosphate, 3 for pyruvate, 2 for acetyl-CoA, 5 for α-ketoglutarate, 1 for CO_2. That is a pleasing confirmation that the carbon skeleton is consistent, and a clear statement of what the lumping cost: a model like this is fit for carbon and energy accounting and would fail a real quality report on consistency grounds.

Blocked reactions — reactions that cannot carry flux anywhere in the feasible space, found by maximising and minimising each one without fixing the objective. The check found exactly one: a phosphate exchange reaction left behind when I removed phosphate from the model's bookkeeping. That is a genuine find in my own work, and it is the typical kind: a vestigial reaction from an edit, harmless to the predictions and a sign that the model and the edit history have diverged. In a draft model, blocked reactions are usually in the hundreds, and they mark pathways whose other half is missing — the gaps that gap filling is for.

Gap filling is a hypothesis, demonstrated

Automated gap filling adds reactions from a universal database until biomass can be produced, choosing the smallest set that does it. To see what that means, I deleted enolase from the model — which chapter 35 found lethal — and offered a candidate pool of six reactions: the true gap, a plausible shortcut (3-phosphoglycerate straight to pyruvate), PEP synthetase, malic enzyme, a decoy citrate lyase and acetate uptake.

Candidate added	Growth restored
Enolase — the true gap	**0.754 h^{-1}**, the original rate
Each of the other five, alone	0 — still infeasible
PEP synthetase *and* the 3-phosphoglycerate-to-pyruvate shortcut, together	**0.723 h^{-1}**

So a minimal-set gap filler, given this pool, would have two defensible answers: one reaction that is correct, and a pair that restores 96% of the growth by a route the organism may not have at all. If the true gap happened to be missing from the universal database — which for a non-model organism's unusual chemistry it often is — the filler would return the pair, the model would grow, and the model would be wrong in a way that no further simulation could reveal.

Hence the discipline the week asks for: list the reactions gap filling added, look each one up for evidence in *this* organism — a gene with a credible annotation, an enzyme assay, a growth phenotype — and treat every one without evidence as **a prediction to test, not a finding**. A gap-filled reaction is a claim that the organism has an enzyme; it is testable by assay, by growth on a diagnostic substrate, or by looking harder at the genome, and reporting the list is what lets a reader judge the model.

The biomass objective, and how wrong it can be

The biomass equation is a set of measured coefficients: macromolecular composition from a dry-weight analysis, amino acid and nucleotide fractions from the protein and nucleic acid content, lipid from the membrane composition, and the ATP requirement from polymerisation costs plus a fitted growth-associated maintenance term. For a non-model organism, almost none of that has been measured, and the universal practice is to copy a related organism's equation.

How much that matters depends on the question. For a **yield ceiling** it matters little: the carbon content of biomass is similar across bacteria, so the ceiling moves by a few per cent. For an **ATP-limited prediction** it matters a great deal, since the growth-associated maintenance coefficient is fitted and can differ twofold. And for a **gene-essentiality prediction** it is decisive, because essentiality is usually decided by whether a specific biomass component can be made — so a component that this organism does not actually require makes its synthesis pathway spuriously essential, and a component it does require but the equation omits makes a real essential gene look dispensable. My transketolase result in chapter 35 is the first of those failure modes in miniature.

Which failures change a prediction

Finding	Changes a flux prediction?	What to do
Mass or charge imbalance in a reaction carrying flux	Yes — it can create or destroy matter, and FBA will exploit it	Fix it; this is the first thing to repair

Stoichiometric inconsistency	Yes, if it permits a mass-generating cycle	Find the offending reactions; exempt deliberate currency lumping and say so
Reaction reversible when it should not be	Yes — loops and impossible routes	Fix directions from thermodynamics (chapter 36)
Blocked reactions	No, by definition — but they mark gaps	Investigate the pathway they belong to; delete vestigial ones
Missing or wrong annotations and identifiers	No	Fix before publication: it is what makes the model reusable, not what makes it right
SBML conformance warnings	No	Fix, so other tools can read it
No growth on a medium the organism grows on	Yes — the model is incomplete	Find whether the medium definition or the network is at fault, in that order

The three I would fix in my own model, with reasons: the stoichiometric inconsistency, by documenting the currency lumping explicitly in the model's notes so that nobody mistakes it for an error, since fixing it properly means adding ADP, phosphate and the oxidised cofactors; the blocked phosphate exchange, by deleting it, which takes a second; and the aerobic formate secretion from chapter 35, by bounding pyruvate formate-lyase to zero in aerobic simulations, which is a real curation decision and should be recorded with its justification. The ones I would leave: annotations, which this model does not have and does not need for a teaching purpose, and the absence of compartments, which is a deliberate scope limit.

When is a draft good enough?

For a **yield ceiling or a stoichiometric feasibility question**, often yes: those depend on the network's carbon and cofactor topology, which a draft usually gets right for central metabolism, and the answer is robust to peripheral errors. For a **gene-deletion prediction**, usually not: essentiality depends on isozymes, on gene–protein–reaction rules, and on the biomass equation, all of which are exactly what a draft gets wrong. In between — a flux map, a by-product prediction, a medium requirement — judge by whether the prediction depends on the parts of the model that were curated or the parts that were inferred, and say which in the paper.

Scale	`core ~40 reactions · draft ~1,000–1,500 · curated ~2,600`
Mass balance	`Σ coefficient × element count = 0 for every internal reaction`
Consistency	`∃ m > 0 with Sᵀm = 0; currency lumping breaks it by design`
Blocked	`max |v| = 0 over the whole feasible space; marks gaps, not errors`
Gap filling	`smallest set restoring biomass; here one right answer and one wrong pair at 96% of growth`
Good enough for	`yield ceilings often; gene essentiality rarely`

Your draft model of a non-model bacterium does not grow on the defined medium the organism grows on. How do you find out whether the medium or the model is at fault?

Separate the two by making the medium generous. Open every exchange reaction — let the model take up everything — and optimise. If it still does not grow, the network is at fault and the problem is a gap in biomass precursor synthesis; find which precursor by setting the biomass reaction aside and maximising the production of each component in turn, which localises the gap to one branch. If it does grow with everything open, the medium definition is at fault, and the way to find the missing component is to add the defined medium's exchanges back one at a time, or to remove them from the open medium one at a time until growth fails.

Then the two causes of a medium problem, which need different fixes: an exchange reaction that does not exist in the model for a nutrient that is in the medium, which is a naming or an annotation problem; and a transport reaction missing between the extracellular and cytosolic compartments, which is a real gap and a common one, since automated reconstruction handles transport worse than it handles metabolism. The second is worth checking first in any draft that will not grow.

Three things people get wrong

- ‘The model passed quality control, so it is right.’ The checks are necessary, not sufficient: consistency, balance and annotation say nothing about whether the organism has these reactions.
- ‘Gap filling completed the model.’ It proposed the smallest set that makes biomass producible; in my test a wrong pair restored 96% of growth.
- ‘A draft is fine for predicting deletions.’ Essentiality is the prediction most sensitive to isozymes, gene rules and biomass composition — the three things a draft gets wrong.

Self-check — answer on paper first

1. Why does a carbon-balance check pass on a model that fails stoichiometric consistency?

Because the carbon check only verifies that carbon is conserved reaction by reaction, while consistency asks for a single positive mass assignment that makes *every* element conserved simultaneously across the whole network. A model that tracks ATP or NADH as bare currency metabolites — with no ADP, phosphate or oxidised partner — conserves carbon perfectly and cannot conserve mass, because those currencies appear on one side of reactions with nothing to balance them. The practical reading is that a consistency failure is only a bug if it is unintentional; deliberate lumping should be documented so that a reader can tell the difference.

2. A gap filler adds three reactions to your draft. What do you do before using the model?

Look each one up for evidence in this organism: a gene with a credible annotation, a characterised enzyme in a close relative, or a growth phenotype that implies the activity. Report all three in the paper as gap-filled, with the evidence or its absence. Then test whether your conclusions depend on them — delete each and see whether the prediction you care about survives — because a conclusion that rests on an unsupported gap-filled reaction is a conclusion about the filler's database, not about the organism.

3. Which quality-report failures would you fix before computing a theoretical yield, and which would you leave?

Fix anything that lets the model create or destroy matter or energy: mass and charge imbalances in reactions that can carry flux, inconsistencies that permit a mass-generating cycle, and reversibility errors that open thermodynamically impossible routes — all of these can inflate a yield. Leave annotation coverage, identifier mapping and SBML conformance, which affect reusability and not the number; leave blocked reactions, which cannot carry flux by definition; and leave compartment metadata unless the yield depends on transport. Then state the biomass equation's provenance, because for a yield the carbon content of biomass is the assumption that matters most.

What to open a book for. Thiele & Palsson, Nat. Protoc. (2010) for what manual reconstruction involves — skim it to understand why automation exists. Machado et al., Nucleic Acids Res. (2018) for CarveMe, Zimmermann et al., Genome Biol. (2021) for gapseq, and Lieven et al., Nat. Biotechnol. (2020) for MEMOTE, read for what the checks are rather than how to run them.

MATLAB and the COBRA Toolbox: reading other people's code

Half the metabolic modelling literature ships MATLAB. You do not need to like the language; you need to be able to read a script, run it, change one thing, and reconcile its answer with yours.

Why this week exists

The COBRA Toolbox came first, and a great deal of published strain-design work — OptKnock and its descendants in week 39, ^{13}C flux analysis packages in week 43, many supplementary scripts — exists only as MATLAB. Being unable to read it means being unable to reproduce it. This chapter is written for someone who already writes Python, so it is organised as a translation rather than a tutorial. I could not install MATLAB or Octave in this environment, so every snippet here is **not run**: treat them as the shape of the code, and check the function signatures against the current Toolbox documentation.

MATLAB for a Python user

	Python / NumPy	MATLAB
Indexing	`v[0]`, zero-based, `v[-1]` for the last	`v(1)`, one-based, `v(end)` for the last
Default type	Arrays are objects; scalars are scalars	Everything is a matrix; a scalar is 1×1
Element-wise	`a * b` is element-wise; `a @ b` is matrix product	`a .* b` is element-wise; `a * b` is matrix product
Solve A·x = b	`np.linalg.solve(A, b)`	`A \ b`
Null space	`scipy.linalg.null_space(S)`	`null(S)`
Least squares	`np.linalg.lstsq(A, b, rcond=None)`	`A \ b` (same operator, overdetermined)
Structured data	dict, DataFrame	struct, cell array, table
Functions	Any file, any number per file	A script runs; a function needs its own file or a `function` block at the end
Stiff ODEs	`solve_ivp(..., method='LSODA')`	`ode15s` (`ode45` for non-stiff)

Two traps beyond syntax. **Row against column vectors**: MATLAB distinguishes 1×n from n×1 and many functions require one or the other, so a dimension mismatch is the first thing to check when a script fails. And **copy-on-write**: MATLAB passes by value, so modifying a model struct inside a function does not change the caller's copy — the opposite of Python's behaviour with mutable objects, and the reason Toolbox functions return a modified model rather than mutating it. That difference is also a small mercy: it makes MATLAB's equivalent of chapter 36's context-manager problem less dangerous, because `changeRxnBounds` returns a new struct instead of editing the original in place.

The translation exercise the week sets is the part 2 chapter 12 linear-algebra script. The same four calls, the same answers:

```
% MATLAB — not run here
S    = [ 1 -1 -1  0  0 ;  0  1  0 -1  0 ;  0  0  1  0 -1 ];
rankS = rank(S);              % 3
N     = null(S);              % 5x2 basis of the null space
v     = A \ b;                % in place of np.linalg.solve(A, b)
fprintf('rank %d, nullity %d\n', rankS, size(N,2));
```

The Toolbox, and the same analyses under other names

Installation is `initCobraToolbox`, which also chooses a solver; with no Gurobi academic licence, `changeCobraSolver('glpk','LP')` is the fallback and is adequate for everything in these weeks. Loading a model gives a struct whose fields are the linear programme of chapter 35, which is worth inspecting once because it makes the correspondence concrete: `S` the stoichiometric matrix, `lb` and `ub` the bounds, `c` the objective vector, `rxns`, `mets` and `genes` the names, `rules` or `grRules` the gene–protein–reaction logic.

COBRApy	COBRA Toolbox
`cobra.io.read_sbml_model(path)`	`readCbModel(path)`

`model.optimize()`	`optimizeCbModel(model)`
`model.objective = 'RXN'`	`changeObjective(model, 'RXN')`
`model.reactions.RXN.bounds = (lo, hi)`	`changeRxnBounds(model, 'RXN', lo, 'l')`
`single_gene_deletion(model)`	`singleGeneDeletion(model)`
`flux_variability_analysis(model, fraction_of_optimum=0.99)`	`fluxVariability(model, 99)`
`cobra.io.write_sbml_model(model, path)`	`writeCbModel(model, 'fileName', path)`

One difference catches everyone: `fluxVariability` takes the fraction as a **percentage** (99) where COBRApy takes a fraction (0.99). A result that looks absurdly tight or loose compared with your Python run is usually this.

The reconciliation exercise is the valuable one. Put the COBRApy and Toolbox results side by side and account for every difference. In order of likelihood: the **medium** — a different default for an exchange bound, especially oxygen; the **objective** — different biomass reaction variants exist in the same model file, one with and one without the growth-associated maintenance term; a **bound** — the glucose uptake default differs between distributions of the same model; and only then the **solver**, which matters for which alternative optimum is returned but not for the objective value. The rule to carry: a mismatch in the objective value is a model difference, and a mismatch in individual fluxes with the same objective value is alternative optima.

ODEs, and where Simulink fits

For the kinetic model of part 2 chapter 16, the translation is direct. MATLAB's stiff solver is `ode15s` and its non-stiff workhorse is `ode45`, corresponding to LSODA and RK45; the stiffness argument of part 2 chapter 12 applies identically, and a metabolic model should go to `ode15s` for the same reason it goes to LSODA.

```
% MATLAB — not run here
function dc = rhs(~, c, p)
    v  = rates(c, p);
    dc = [v(1)-v(2); v(2)-v(3); v(3)-v(4)];
end
opts = odeset('RelTol', 1e-10, 'AbsTol', 1e-12);
[t, y] = ode15s(@(t,c) rhs(t, c, p), [0 20000], [0.1; 0.1; 0.1], opts);
```

Note the column vectors — `[0.1; 0.1; 0.1]`, not `[0.1, 0.1, 0.1]` — which is the row-against-column trap in its most common form. The check the week asks for is that the steady state and the flux control coefficients agree to three decimal places with the Python version; if they do not, the suspects are the initial condition's shape, a transcription error in a rate law, and the tolerances, in that order.

Simulink is a block-diagram environment for dynamic systems: you assemble a model from blocks representing integrators, transfer functions, gains and feedback paths, and simulate it. Process and control engineers reach for it because a control loop is naturally a diagram — sensor, controller, actuator, plant — and because it connects to hardware-in-the-loop testing and code generation for real controllers. For metabolic modelling it is rarely used, and knowing what it is for is enough; the one place it genuinely fits is the dynamic-control analysis of part 7 chapter 24, where the question is a loop's stability and settling time.

Crossing platforms deliberately

Write a model to SBML from one platform, read it in the other, and compare reaction counts, metabolite counts, bounds, objective and gene rules. What to expect, with the caveat from chapter 36 that I could not test it here: counts, bounds and the objective survive, carried by the FBC extension; gene rules usually survive but their formatting differs, so a string comparison will show spurious differences; annotations, notes and anything stored in free-text fields are the casualties. The direction that is safer is generally the one with fewer custom attributes — a model freshly read from BiGG travels well in either direction, and a model that has been manipulated by a script with its own conventions does not.

Committing both versions of the same analysis with a README explaining why both exist is a real portfolio item: it says you can work in the toolchain the field actually uses, which includes code older than you are.

Indexing	`MATLAB is one-based; end is the last element`
Solve	`A \ b for both determined and least-squares systems`
FVA argument	`Toolbox takes 99 (per cent), COBRApy takes 0.99 (fraction)`
Stiff ODEs	`ode15s ≈ LSODA; column vectors for initial conditions`
	`objective differs → model difference; fluxes differ at equal objective → alternative`

Mismatch rule	`optima`

A published MATLAB script reports a growth rate you cannot reproduce in COBRApy with the same model file. Work through it.

First, print both media. `model.summary()` in COBRApy and the exchange bounds from the Toolbox struct — the most common cause is a different default on oxygen or on glucose uptake, and the second most common is an exchange the script opened with `changeRxnBounds` somewhere earlier in the file. Second, check which reaction is the objective: many model files contain two biomass reactions, with and without the growth-associated maintenance term, and they give noticeably different growth rates. Third, check the maintenance reaction's lower bound, which some distributions set to zero and others to the fitted value.

If the objective values then agree but individual fluxes do not, stop worrying: that is alternative optima, and the two solvers have picked different vertices of the same optimal face. Confirm it by running pFBA in both, which selects among them by a stated criterion and should bring them close. If the objective values still differ after the medium, the objective and the maintenance bound match, suspect the model file itself — compare reaction counts, and check whether one platform silently dropped reactions on import, which is the SBML round-trip problem from the other direction.

Three things people get wrong

- ‘The solvers disagree.’ They almost never do on the objective value. Check the medium, the objective reaction and the maintenance bound first.
- ‘`fluxVariability(model, 0.99)`.’ The Toolbox wants 99. A tiny argument difference, a wildly different answer.
- ‘I will avoid MATLAB.’ Then you cannot reproduce a large fraction of the strain-design and flux analysis literature.

Self-check — answer on paper first

1. Translate: `np.linalg.solve(A, b)`, `null_space(S)`, `v[-1]`, `a * b` element-wise.

`A \ b`; `null(S)`; `v(end)`; `a .* b`. And note that `A \ b` also covers the over-determined case that needs `lstsq` in NumPy, which is convenient and occasionally hides the fact that you are solving a least-squares problem rather than an exact one.

2. Your Toolbox and COBRApy runs give the same growth rate and different acetate secretion. What is happening and how do you confirm it?

Alternative optima: the optimal face of the linear programme contains more than one vertex, and the two solvers have returned different ones, which chapter 35 found for fourteen reactions in a 40-reaction model. Confirm it by running flux variability analysis on the acetate exchange at 100% of the optimum in either platform — if its range spans both values, the difference is a property of the model and not of the software. Then, if you need a single reportable number, use pFBA in both, and report the range alongside it.

3. Why does modifying a model inside a MATLAB function not affect the caller's model, and what is the Python equivalent danger?

Because MATLAB passes arguments by value with copy-on-write, so a function receives its own copy and must return the modified struct for the change to persist — which is why Toolbox functions are written as `model = changeRxnBounds(model, ...)`. The Python danger is the mirror image: COBRApy model objects are mutable and passed by reference, so a function that knocks out a reaction changes the caller's model permanently, which is exactly the accumulating-knockout bug of chapter 36 and the reason for the `with model:` context manager.

What to open a book for. The COBRA Toolbox documentation and installation guide; MATLAB's getting-started material, read with the translation table above in hand; GNU Octave if you have no licence, which runs most of what you need outside the Toolbox itself; and the MATLAB ODE solver documentation for `ode45` and `ode15s`.

Closing notes to part 10

The four weeks in one page

Week 35. FBA is max $c^{T}v$ subject to $S \cdot v = 0$ and bounds. A 40-reaction core model has 11 degrees of freedom, grows at 0.754 h^{-1} aerobically and 0.217 anaerobically, yields 0.42 g biomass per g glucose, and needs 0.26 $mmol \cdot gDW^{-1} \cdot h^{-1}$ of glucose before it grows at all. Ten of thirty-nine deletions are lethal; the pentose-phosphate detour forced by deleting phosphoglucose isomerase costs a third of the growth rate. Glucose's shadow price is 0.0754 and oxygen's is zero, because oxygen is not binding.

Week 36. Succinate dehydrogenase and fumarate reductase together form a cycle that satisfies the mass balances exactly and cannot happen; flux variability shows both running to the 1,000 bound. pFBA removes it and cuts total flux from 172.9 to 154.4 at identical growth; loopless FBA removes it on principle. Thermodynamics becomes a bound when the sign of $\Delta G'$ is fixed across the feasible concentration range. Use the medium, not bounds, and the context manager, not accumulated knockouts.

Week 37. The checks found three things in my own model: all 28 internal reactions carbon-balanced with biomass draining 70.87 C per unit; stoichiometric consistency failing on the whole model and passing on the carbon subnetwork, where the recovered mass vector is exactly the carbon counts; and one blocked reaction, a vestigial phosphate exchange. Deleting enolase and offering a candidate pool, the true gap restored growth alone and a wrong pair restored 96% of it — gap filling is a hypothesis.

Week 38. A translation table, the Toolbox's percentage-versus-fraction trap, and the reconciliation rule: a different objective value is a model difference, the same objective value with different fluxes is alternative optima.

On the numbers in this part

Every figure came from the scripts described: a 40-reaction model built by hand, solved with `scipy.optimize.linprog`, with flux variability, deletions, pFBA, shadow prices and the quality checks implemented directly. Four things to hold against them. The model is mine, not the published core model, so its aerobic growth of 0.754 differs from the tracker's reference 0.87 — the anaerobic figure agrees. The P/O assumptions, 2.5 ATP per NADH and 1.5 per quinol, set the ATP yield of 32 per glucose and therefore the growth rate. The biomass coefficients are rounded from the core model's and the carbon content, 70.87 per unit, sets the yield ceiling. And the draft-model percentages in figure 37.1's left panel are illustrative rather than measured, and are labelled so on the figure.

COBRApy, MEMOTE, CarveMe, gapseq, MATLAB and Octave could not be installed here, so every snippet involving them is marked not run. The three errors I made while building the model — phosphate bookkeeping, the missing soluble transhydrogenase, and the vestigial exchange — are described where they occurred rather than quietly fixed, because each is the kind of error a package hides and a hand-built model forces you to find.

What remains

Written so far: weeks 1 to 38 and 45 to 61 — 55 of the tracker's 66. Remaining: weeks 39 to 44 and 62 to 66. The next block is strain-design algorithms, retrosynthesis and machine learning, Bayesian optimisation, ^{13}C flux analysis and the fermentation and scale-up weeks (39 to 44) — which is where the modelling of this part becomes a design tool and where part 3's transport arithmetic returns. Then the five professional weeks. I will take weeks 39 to 44 next.